

Cahier des charges détaillé

Assistant conversationnel métier pour la consultation, le filtrage et l'aide à la décision

Finalité du projet

Permettre aux utilisateurs d'interroger les projets, gammes, validations et résultats en français, avec des réponses fiables, sécurisées et vérifiables.

Version : 1.0

Date : 24 juillet 2026

Public : équipe projet, développeurs, responsables métier et futurs utilisateurs

Niveau : document adapté à un débutant en intelligence artificielle

Sommaire

1. Contexte et vision

2. Objectifs et périmètre

3. Utilisateurs et droits

4. Parcours utilisateurs

5. Exigences fonctionnelles

6. Architecture cible

7. Outils et technologies

8. Données et modèles

9. Outils métier du chatbot

10. API à développer

11. Interface React

12. Étapes de réalisation

13. Sécurité et confidentialité

14. Stratégie de tests

15. Performance et supervision

16. Critères d'acceptation

17. Planning indicatif

18. Évolution vers l'aide à la décision

19. Livrables et validation

20. Glossaire pour débutant

1. Contexte et vision

La plateforme actuelle permet de gérer des utilisateurs, projets, gammes, fichiers Excel, validations par step, commentaires, notifications, KPI et journaux d'audit. L'information est disponible mais répartie entre plusieurs pages et tableaux.

Le projet consiste à ajouter un assistant conversationnel métier. L'utilisateur pose une question en français ; l'assistant comprend la demande, recherche les données autorisées dans la plateforme et répond de façon claire.

Exemple attendu : « La gamme Freinage V2 est en cours de validation à 78 %. Elle contient 3 NOK et sa date de fin prévue est le 12 août 2026. »

L'assistant n'est pas un accès direct à la base de données. Il utilise uniquement des fonctions Django contrôlées. Cette séparation garantit la fiabilité, les permissions et la traçabilité.

2. Objectifs et périmètre

2.1 Objectif principal

Permettre aux utilisateurs de consulter et filtrer les informations métier en langage naturel, sans devoir connaître la structure technique de l'application.

2.2 Objectifs secondaires

- Réduire le temps nécessaire pour trouver une information.
- Faciliter l'utilisation par des profils non techniques.
- Résumer l'état d'une gamme ou d'un projet.
- Afficher les NOK, retards, steps restants et personnes affectées.
- Conserver le contexte d'une conversation.
- Préparer une évolution vers des recommandations décisionnelles.

2.3 Périmètre du MVP

Le MVP est une première version utile et sécurisée. Il est exclusivement en lecture. Il couvre cinq capacités :

1. Rechercher une gamme.
2. Afficher son état et sa progression.
3. Afficher ses NOK.
4. Afficher les gammes en retard.
5. Résumer l'état d'un projet.

2.4 Hors périmètre du MVP

- Modifier une gamme ou un projet.
- Affecter automatiquement un valideur.
- Changer un statut ou clôturer une validation.
- Envoyer automatiquement un message ou un e-mail.
- Entraîner un modèle de Deep Learning personnalisé.
- Exécuter une requête SQL générée par un modèle.

3. Utilisateurs et droits

Profil	Données consultables	Limites
Administrateur	Tous les projets, gammes, utilisateurs, KPI et audits.	Aucune limite fonctionnelle de lecture.
PPL	Projets auxquels il est affecté, gammes, progression, résultats et valideurs.	Aucun accès aux projets non affectés.
Valideur	Gammes accessibles, steps, cotations, commentaires et tâches.	Aucun accès aux projets non affectés.
Visiteur	Informations de lecture autorisées sur ses projets.	Aucune modification et visibilité limitée.

Exigence critique : le chatbot doit utiliser l'utilisateur authentifié côté Django. Il ne doit jamais faire confiance à un rôle ou un identifiant transmis uniquement par le frontend.

4. Parcours utilisateurs

4.1 Consultation directe

Utilisateur : Quel est l'état de la gamme Freinage V2 ?

Assistant :

La gamme Freinage V2 est en cours de validation.

Progression : 78 %

Steps terminés : 94 sur 120

NOK : 3

Date de fin prévue : 12 août 2026

[Ouvrir la gamme] [Voir les NOK] [Voir le rapport]

4.2 Résultat ambigu

Utilisateur : Montre-moi la gamme freinage.

Assistant :

J'ai trouvé trois gammes :

1. Freinage V1 – Projet Atlas
2. Freinage V2 – Projet Atlas
3. Freinage Hybrid – Projet Orion

Laquelle souhaitez-vous consulter ?

4.3 Conversation avec contexte

Utilisateur : Affiche les gammes du projet Atlas.

Assistant : 14 gammes trouvées.

Utilisateur : Seulement celles qui sont en retard.

Assistant : 4 gammes sont en retard.

Utilisateur : Parmi elles, affiche celles avec des NOK.

Assistant : 2 gammes correspondent à ces critères.

5. Exigences fonctionnelles

Réf.	Fonction	Description	Priorité
F01	Ouvrir le chatbot	Bouton flottant ou accès dans la navigation.	Obligatoire
F02	Envoyer une question	Saisie en français avec chargement et gestion d'erreur.	Obligatoire
F03	Rechercher une gamme	Par identifiant, nom, projet, VIN, CMQ, fonction ou fichier.	Obligatoire
F04	Afficher l'état	Statut, progression, cotations, dates, retard et responsables.	Obligatoire
F05	Afficher les NOK	Filtrer par gamme, projet, EV, step, cotation et période.	Obligatoire
F06	Filtrer les gammes	Par statut, retard, progression, projet ou valideur.	Obligatoire
F07	Résumer un projet	Avancement, volumes, retards, NOK et risques.	Obligatoire
F08	Gérer l'ambiguïté	Proposer une liste lorsque plusieurs résultats correspondent.	Obligatoire
F09	Conserver le contexte	Comprendre « celle-ci », « la première » ou « ce projet ».	Importante
F10	Proposer des liens	Boutons vers la gamme, le rapport ou le projet.	Importante
F11	Collecter un feedback	Boutons « utile » et « incorrecte » avec commentaire.	Importante
F12	Historique	Consulter et supprimer ses propres conversations.	Souhaitable

5.1 Informations d'une gamme

- Nom, projet, véhicule, architecture, motorisation et fonction.
- Statut métier et pourcentage de progression.
- Steps totaux, terminés et restants.
- Répartition des cotations : OK, NOK, NOK mineur et à coter.
- Dates planifiées et réelles, avec retard éventuel.
- Utilisateurs et rôles affectés.
- Lien vers les pages correspondantes.

5.2 Gestion des erreurs

L'assistant doit signaler clairement : résultat absent, résultat ambigu, accès interdit, donnée manquante, service IA indisponible, demande non comprise ou question hors périmètre.

6. Architecture cible



Règle d'architecture : le modèle de langage choisit un outil et formule une réponse. Django contrôle les permissions, exécute la requête métier et reste l'unique source de vérité.

6.1 Question sur une donnée actuelle

Utiliser les fonctions Django et PostgreSQL : état, progression, retards, affectations et résultats.

6.2 Question sur un document

Une évolution RAG pourra rechercher dans le guide utilisateur et les procédures. Cette partie n'est pas nécessaire pour le premier MVP.

7. Outils et technologies

7.1 Outils déjà présents

Outil	Rôle
Django	Backend, règles métier, permissions et orchestration.
Django REST Framework	Endpoints HTTP et validation des requêtes.
PostgreSQL	Stockage des données métier et conversations.
React	Interface conversationnelle.
Axios	Communication entre React et Django.
JWT	Authentification de l'utilisateur.
Celery et Redis	Traitements longs, cache et tâches asynchrones.
Docker	Environnement reproductible et déploiement.
Vitest et tests Django	Validation automatique du frontend et du backend.

7.2 Outils à ajouter

Outil	Utilité	Moment
SDK du fournisseur LLM	Comprendre la question et appeler une fonction.	MVP
Validation JSON/Pydantic ou serializers DRF	Contrôler les paramètres produits par le modèle.	MVP
Extension PostgreSQL pg_trgm	Recherche tolérante aux petites fautes.	MVP ou phase 2
pgvector	Recherche sémantique dans commentaires et documents.	Phase 3
Suivi centralisé des erreurs	Détecter les erreurs backend et frontend.	Avant production
MLflow	Versionner les futurs modèles prédictifs.	Phase décisionnelle

7.3 Choix d'hébergement du modèle

Le choix doit être validé par la politique de sécurité de l'entreprise :

- **API approuvée** : déploiement rapide, infrastructure limitée.
- **Modèle local** : davantage de contrôle, mais besoin de serveurs et de maintenance.

- **Hybride** : données sensibles localement et utilisation externe limitée à des données filtrées.

8. Données et modèles

8.1 Conversation

```
Conversation
- id : UUID
- user : utilisateur propriétaire
- title : titre court
- context : JSON
- created_at
- updated_at
```

8.2 Message

```
ConversationMessage
- id
- conversation
- role : USER, ASSISTANT ou TOOL
- content
- tool_name
- tool_arguments : JSON
- tool_result : JSON
- execution_time_ms
- created_at
```

8.3 Feedback

```
AssistantFeedback
- id
- message
- user
- useful : oui / non
- comment
- created_at
```

8.4 Contexte structuré

```
{
  "selected_project_id": 7,
  "selected_gamme_id": 42,
  "active_filters": {
    "delayed": true,
    "has_nok": true
  }
}
```

Les données sensibles ne doivent pas être conservées inutilement dans les conversations. Une durée de conservation et un mécanisme de suppression doivent être définis.

9. Outils métier du chatbot

Un « outil » est une fonction backend autorisée que le modèle peut demander à Django d'exécuter. Chaque outil reçoit obligatoirement l'utilisateur authentifié.

Outil	Fonction	Paramètres principaux
search_gammes	Rechercher et filtrer des gammes accessibles.	texte, projet, statut, retard, NOK
get_gamme_status	Retourner l'état complet d'une gamme.	gamme_id ou nom
get_gamme_nok	Afficher les cotations NOK et NOK mineur.	gamme_id, EV, step
get_project_summary	Résumer l'avancement d'un projet.	project_id ou nom
get_delayed_gammes	Lister les gammes en retard.	projet, retard minimal

9.1 Exemple de sélection d'un outil

Question :
"Quel est l'état de la gamme Freinage du projet Atlas ?"

Décision structurée du modèle :

```
{
  "tool": "get_gamme_status",
  "arguments": {
    "gamme_name": "Freinage",
    "project_name": "Atlas"
  }
}
```

Django valide ensuite les paramètres, recherche uniquement dans les projets accessibles et retourne un résultat structuré au modèle.

10. API à développer

Méthode	Endpoint	Fonction
POST	/admin_config/assistant/message/	Envoyer une question.
POST	/admin_config/assistant/conversations/	Créer une conversation.
GET	/admin_config/assistant/conversations/	Lister ses conversations.
GET	/admin_config/assistant/conversations/{id}/	Afficher une conversation.
DELETE	/admin_config/assistant/conversations/{id}/	Supprimer sa conversation.
POST	/admin_config/assistant/feedback/	Évaluer une réponse.

10.1 Exemple de requête

```
{
  "conversation_id": "7be8b86c-4cbe-42ab-823d-b842fc030857",
  "message": "Je veux voir l'état de la gamme X"
}
```

10.2 Exemple de réponse

```
{
  "answer": "La gamme X est en cours de validation à 78 %.",
  "intent": "GET_GAMME_STATUS",
  "data": {
    "gamme_id": 42,
    "status": "EN_VALIDATION",
    "progress": 78
  },
  "links": [{
    "label": "Ouvrir la gamme",
    "url": "/validation/42"
  }],
  "suggestions": [
    "Afficher les NOK",
    "Voir les steps restants"
  ]
}
```

11. Interface React

11.1 Structure proposée

```
Frontend/src/components/assistant/  
├─ AssistantButton.jsx  
├─ AssistantPanel.jsx  
├─ MessageList.jsx  
├─ MessageBubble.jsx  
├─ ResultCard.jsx  
├─ GammeCard.jsx  
├─ ProjectSummaryCard.jsx  
└─ SuggestedQuestions.jsx  
  
Frontend/src/hooks/  
└─ useAssistant.js
```

11.2 Éléments visuels

- Bouton flottant disponible sur les pages authentifiées.
- Panneau latéral adapté à l'ordinateur et à la tablette.
- Messages utilisateur et assistant clairement différenciés.
- Cartes pour les gammes, projets et NOK.
- Indicateur de chargement et bouton d'annulation.
- Questions suggérées au démarrage.
- Boutons « ouvrir », « voir les NOK », « voir le rapport ».
- Feedback « utile » ou « incorrecte ».

12. Étapes détaillées de réalisation

Étape 0 — Apprentissage des bases

Apprendre progressivement : API REST, JSON, requêtes HTTP, modèles Django, QuerySets, serializers DRF, composants React, hooks React, JWT et principe d'un LLM.

Important : il n'est pas nécessaire de connaître le Deep Learning pour développer le premier MVP.

Étape 1 — Formaliser les règles métier

- Définir une gamme commencée, terminée et en retard.
- Définir la formule de progression.
- Définir les cotations finales et les NOK.
- Définir les données visibles par chaque rôle.

Livrable : document des règles métier validé.

Étape 2 — Sécuriser et centraliser le backend

- Corriger les permissions des référentiels.
- Valider les changements de statut.
- Centraliser le calcul de progression et de retard.
- Ajouter une matrice de tests des rôles.

Livrable : services métier fiables et testés.

Étape 3 — Développer les cinq outils sans IA

Implémenter et tester les fonctions de recherche, état, NOK, retards et résumé de projet. Elles doivent d'abord retourner du JSON correct sans utiliser de modèle.

Livrable : cinq fonctions Python sécurisées.

Étape 4 — Créer l'API conversationnelle

- Créer les modèles Conversation, Message et Feedback.
- Créer serializers, endpoints et permissions.
- Ajouter la gestion uniforme des erreurs.
- Limiter la longueur et la fréquence des requêtes.

Étape 5 — Construire l'interface React

Construire le panneau de chat, l'envoi de message, l'affichage des cartes et les liens. Utiliser d'abord des réponses simulées pour valider l'ergonomie.

Étape 6 — Connecter le modèle de langage

- Ajouter le SDK uniquement côté backend.
- Déclarer la liste fermée des outils disponibles.
- Valider systématiquement les paramètres.
- Demander une précision en cas d'ambiguïté.
- Formuler la réponse à partir du seul résultat Django.

Étape 7 — Ajouter le contexte conversationnel

Conserver le projet, la gamme et les filtres sélectionnés. Éviter d'envoyer systématiquement tout l'historique au modèle.

Étape 8 — Améliorer la recherche textuelle

1. Identifiant exact.
2. Nom exact, sans sensibilité à la casse.
3. Nom partiel.
4. Similarité trigramme.
5. Recherche vectorielle dans une phase ultérieure.

Étape 9 — Tester la sécurité et les réponses

Tester tous les rôles, les projets non affectés, les noms ambigus, les fautes, les questions hors sujet et les tentatives de contournement.

Étape 10 — Déployer un pilote

Commencer avec un petit groupe : un administrateur, un PPL, deux valideurs et un visiteur. Collecter leurs retours avant un déploiement général.

13. Sécurité et confidentialité

- ☐ Authentification obligatoire pour tous les endpoints.
- ☐ Permissions vérifiées dans chaque outil métier.
- ☐ Aucune clé API dans React ou dans Git.
- ☐ Liste fermée des outils autorisés.
- ☐ Aucune exécution de SQL généré par le modèle.
- ☐ Validation stricte des paramètres JSON.
- ☐ Limitation de fréquence et de longueur des messages.
- ☐ Chiffrement HTTPS en production.
- ☐ Masquage des données personnelles inutiles.
- ☐ Durée de conservation des conversations définie.
- ☐ Journalisation des erreurs sans enregistrer les secrets.

- ❑ Validation de l'hébergement du modèle par l'entreprise.

Les données de l'entreprise ne doivent pas être envoyées à un service externe sans autorisation formelle. Le choix entre API cloud et modèle local est une décision de sécurité, pas seulement une décision technique.

14. Stratégie de tests

14.1 Tests unitaires backend

- Recherche exacte, partielle et ambiguë.
- Calcul de progression et retard.
- Format des résultats des cinq outils.
- Validation des paramètres.
- Gestion des données absentes.

14.2 Matrice de permissions

Situation	Résultat attendu
Administrateur	Accès autorisé à toutes les données de lecture.
PPL affecté	Accès autorisé au projet.
PPL non affecté	Accès refusé.
Valideur affecté	Accès autorisé selon son rôle.
Valideur non affecté	Accès refusé.
Utilisateur non connecté	Réponse HTTP 401.

14.3 Jeu d'évaluation conversationnel

Préparer au moins 100 questions annotées avec l'outil et les paramètres attendus. Inclure des fautes, synonymes, noms incomplets, ambiguïtés, questions interdites et formulations familières.

14.4 Tests frontend et E2E

- Ouverture et fermeture du panneau.
- Envoi, chargement, erreur et nouvel essai.
- Navigation depuis les cartes.
- Conservation du contexte.
- Tests complets des principaux parcours utilisateurs.

15. Performance et supervision

Indicateur	Objectif initial
Recherche PostgreSQL simple	Moins d'une seconde dans les conditions normales.
Réponse complète du chatbot	Idéalement moins de cinq secondes.
Disponibilité	Alignée sur la disponibilité de la plateforme.
Résultats retournés	Limite et pagination obligatoires.
Erreurs	Mesurées et visibles dans un tableau de supervision.

Métriques recommandées : temps de réponse, outil choisi, taux d'erreur, questions incomprises, taux de feedback positif, coût d'utilisation du modèle et volume par rôle.

16. Critères d'acceptation du MVP

- ☐ L'utilisateur authentifié peut ouvrir le chatbot.
- ☐ Il peut demander l'état d'une gamme en français.
- ☐ La recherche fonctionne avec un nom partiel.
- ☐ Le système demande une précision en cas d'ambiguïté.
- ☐ La réponse affiche statut, progression, cotations et dates.
- ☐ L'utilisateur peut filtrer les gammes en retard.
- ☐ L'utilisateur peut afficher les NOK d'une gamme.
- ☐ L'assistant peut résumer un projet.
- ☐ Le contexte est conservé sur plusieurs messages.
- ☐ Les permissions par projet sont respectées.
- ☐ Aucune donnée absente n'est inventée.
- ☐ La réponse fournit un lien vers la donnée correspondante.
- ☐ Les utilisateurs peuvent évaluer les réponses.
- ☐ Les tests critiques backend et frontend réussissent.

17. Planning indicatif pour un débutant

Phase	Contenu	Durée indicative
1. Apprentissage	API, JSON, Django, React et bases des LLM.	2 à 3 semaines
2. Règles métier	Définitions, permissions et calculs.	1 semaine
3. Outils backend	Cinq fonctions métier et tests.	2 semaines
4. API conversationnelle	Modèles, endpoints et permissions.	1 à 2 semaines
5. Interface React	Panneau, messages, cartes et navigation.	1 à 2 semaines
6. Connexion du LLM	Appels d'outils et formulation.	1 à 2 semaines
7. Validation	Sécurité, tests et corrections.	1 à 2 semaines
8. Pilote	Retours réels et stabilisation.	1 semaine

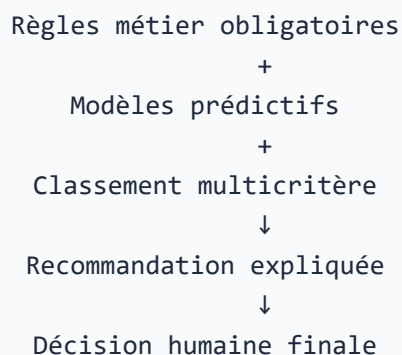
Pour une personne débutante travaillant seule, un MVP sérieux représente environ 10 à 14 semaines. Cette estimation dépend du temps disponible, de la validation métier et du choix d'hébergement du modèle.

18. Évolution vers l'aide à la décision

Une fois la consultation fiable, l'assistant pourra proposer des recommandations :

- gammes à traiter en priorité ;
- causes probables d'un retard ;
- actions nécessaires pour terminer une gamme ;
- score de risque de NOK ;
- date estimée de fin ;
- cas historiques similaires.

18.1 Méthode recommandée



18.2 Exemple

Recommandation : traiter Freinage V2 en priorité.

Raisons :

- deux NOK majeurs non résolus ;
- retard actuel de trois jours ;
- une gamme dépendante est bloquée.

Actions proposées :

1. Retester EV-014 ;
2. Affecter l'anomalie à l'équipe réseau ;
3. Replanifier la fin au 18 août.

Toute recommandation doit afficher ses raisons, son niveau de confiance et les données utilisées. La décision finale reste humaine.

19. Livrables et validation

Livrable	Contenu
L1 — Règles métier	Définitions validées de progression, retard, statut et permissions.
L2 — Backend	Modèles, migrations, outils, orchestrateur, API et tests.
L3 — Frontend	Interface de chat responsive et intégrée.
L4 — Configuration IA	Connexion sécurisée, prompts et schémas d’outils.
L5 — Tests	Tests unitaires, permissions, API, frontend et jeu d’évaluation.
L6 — Documentation	Installation, configuration, utilisation et dépannage.
L7 — Pilote	Rapport de résultats, erreurs et améliorations.

19.1 Validation formelle

Validation métier

Nom :
Date :
Signature :

Validation technique

Nom :
Date :
Signature :

20. Glossaire pour débutant

Terme	Définition simple
API	Moyen standard permettant au frontend de demander des données au backend.
Backend	Partie serveur qui applique les règles et communique avec la base.
Frontend	Interface visible et utilisée dans le navigateur.
JSON	Format texte utilisé pour échanger des données entre applications.
LLM	Modèle capable de comprendre et produire du langage naturel.
Appel d'outil	Demande structurée du modèle pour exécuter une fonction backend autorisée.
RAG	Méthode permettant de rechercher dans des documents avant de répondre.
Embedding	Représentation numérique d'un texte utilisée pour trouver des contenus similaires.
MVP	Première version suffisamment complète pour être testée par de vrais utilisateurs.
Hallucination	Réponse inventée par un modèle alors que l'information n'est pas disponible.
JWT	Jeton utilisé par l'application pour authentifier un utilisateur.
Prompt	Instructions et contexte envoyés au modèle de langage.

Recommandation finale : commencer par les données et les fonctions métier, construire l'API et l'interface, puis connecter le modèle. L'intelligence artificielle facilite la compréhension de la question ; Django reste responsable de la vérité métier et de la sécurité.